

Color Follow (PTZ)

1. Experiment Objective

PTZ pan-tilt + car body collaborative color following — the pan-tilt locks onto the target with PD tracking, the car body automatically turns to home based on the pan-tilt horizontal angle, and distance following is driven by the target's enclosing-circle radius.

Note: the ESP camera's data transmission is affected by the network. A low frame rate degrades the tracking/following effect. If you notice a low frame count, switch the network or use a phone hotspot.

2. Experiment Principle

1. Key Terms

Term	Description
Visual Following	Visual tracking + coordinated body movement; the pan-tilt locks onto the target center while the car actively turns and moves forward/back to follow
Error sharing (body_share)	The body only takes on the pan-tilt horizontal angle $\times$ body_share ratio of the homing task; the remaining error is naturally absorbed by the PTZ, preventing the body and pan-tilt from fighting over control
Angular-rate feedforward (body_ff_gain)	When the body rotates, the pan-tilt compensates in the opposite direction synchronously (canceling $\text{body_az} \times \text{body_ff_gain} / \text{ptz_rate}$ radians per PTZ tick), preventing the target from being dragged away by the body rotation
Distance Follow	PD control based on the target's enclosing-circle radius: $\text{radius} < \text{target_radius} \rightarrow \text{move back}$, $\text{radius} > \text{target_radius} \rightarrow \text{move forward}$
Lost-frame tolerance (lost_frame_threshold)	Only after N consecutive lost frames is the target judged lost and the body frozen, filtering transient frame loss such as lighting flicker
HSV Color Space	A three-dimensional color description of hue/saturation/value; a specified color object is segmented by threshold filtering
PD Control	Proportional-derivative control; the tracking error (P) and its rate of change (D) drive the output, with no integral term to avoid overshoot

2. Technical Architecture

Node chain: state estimation (ekf_filter_node + camera_node) → color_follow_ptz_node → /cmd_ptz_angle (pan-tilt) + /cmd_vel (body)

Three-layer coordination: PTZ PD (30 Hz) + body angular-velocity PD (10 Hz) + distance-follow PD (10 Hz)

3. Core Topics Quick Reference

Firmware Uplink

Topic Name	Message Type	QoS	Source
/camera/image_raw	Image	best_effort	ESP32 camera
/odom	Odometry	best_effort	Odometry (needed by the body control)

Firmware Downlink

Topic Name	Message Type	QoS	Description
/cmd_ptz_angle	Vector3	best_effort + keep_last(1)	x: [-90,90]° horizontal, y: [-90,20]° tilt
/cmd_vel	Twist	best_effort + keep_last(1)	angular.z: turning, linear.x: forward/back

ROS2 Internal

Topic Name	Message Type	QoS	Source	Description
/joint_states	JointState	reliable	State estimation layer	Current pan-tilt angle feedback

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☒
No-PTZ version	☐
Required peripherals	ESP32 camera (pan-tilt) + PTZ servo

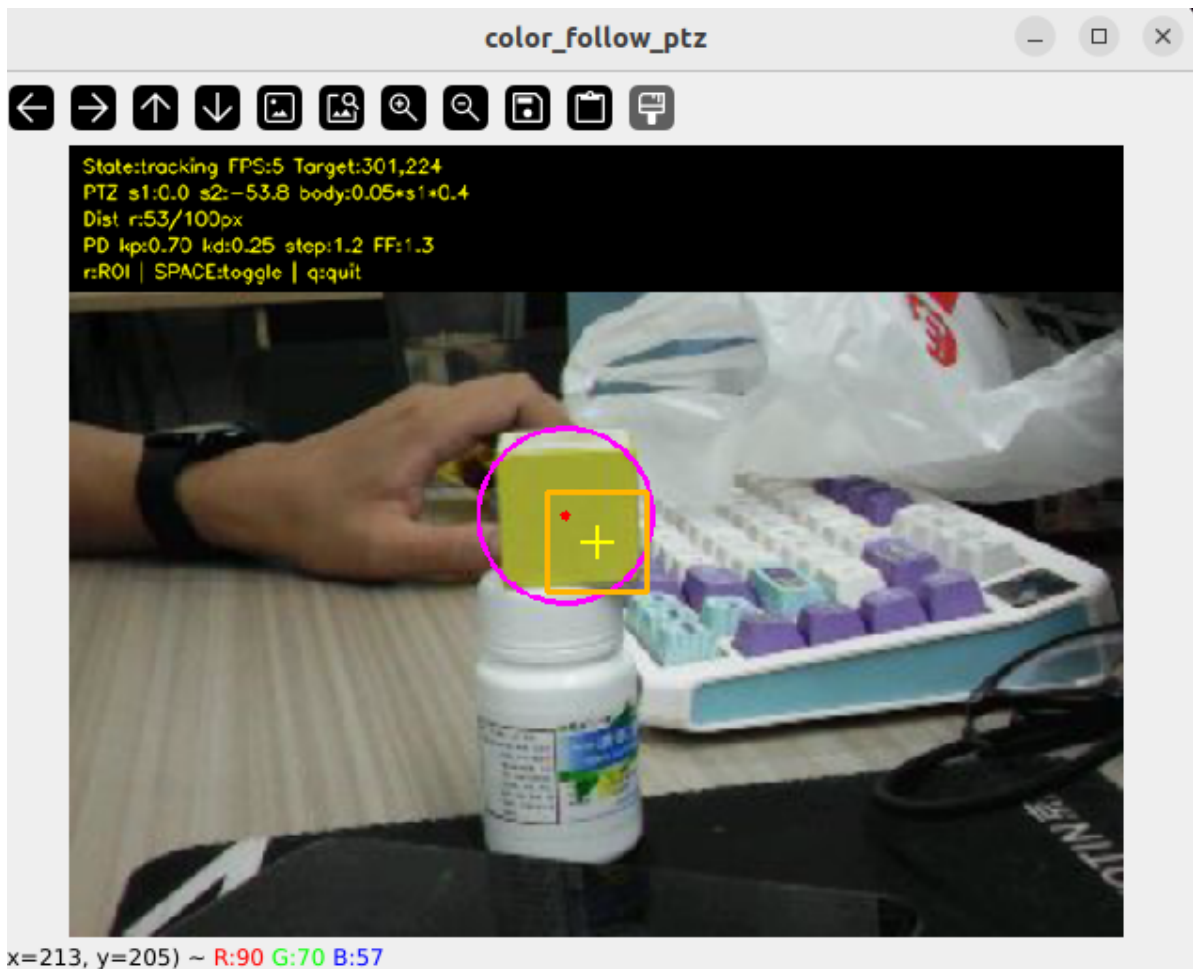
2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Color Follow (Terminal 2)

```
ros2 launch microros_v2_ptz_visual_control_pkg color_follow_ptz.launch.py
```



4. Operation Instructions

Workflow:

After startup, the node processes frames at the image frame rate with two independent control timers running in parallel (PTZ 30 Hz + body 10 Hz):

1. **Image decoding and HSV detection:** the image callback converts the raw Image frame → flips/rotates → scales it to 640×480. BGR is converted directly to HSV, binarized with `cv.inRange` using the six `Hmin~Vmax` parameters, morphological open+close operations remove noise, the largest contour is extracted, and the minimum enclosing circle (cx, cy, r) is computed. An area $\geq$ `min_target_area` (60 px²) and radius $\geq$ `min_target_radius` (5 px) marks the target as valid.
2. **N-frame lost-frame tolerance:** when the target is lost consecutively, the `lost_frames` counter increments; only when it reaches `lost_frame_threshold` (5 frames) is the target truly judged lost and the body frozen.
3. **PTZ PD control (30 Hz):** normalized pixel error → PD step → angular-rate feedforward compensation → step limiting ($\pm 1.2^\circ$) → step smoothing ($\alpha=0.35$) → direction reversal → angle smoothing ($\alpha=0.85$) → publish `/cmd_ptz_angle`.
4. **Body angular-velocity PD control (10 Hz):** takes the PTZ horizontal angle `s1`, zeros it within the angle tolerance (`angular_tolerance_deg=3°`). Multiplied by `body_share` (0.4) to get the effective homing error `s1_eff`, PD computes the angular velocity `angular.z`, published to `/cmd_vel`.
5. **Distance-follow PD control (10 Hz):** enabled with `enable_distance_follow=true`. Computes the deviation between the target radius and `target_radius` (100 px), PD computes the linear velocity, and publishes it merged with the angular velocity to `/cmd_vel`.
6. **Target lost → freeze the body:** the target is lost for >5 consecutive frames → reset all PD states and publish zero-velocity `/cmd_vel`.

Interaction:

Key	Function	Description
Space	idle ↔ tracking	Toggles the follow state; publishing zero velocity to stop in idle
i / I	Back to idle	Stops PTZ and body control and publishes zero velocity
r / R	ROI color pick	Drag a box with the mouse to select the target color region; the HSV threshold is sampled automatically
q / Q / ESC	Exit	Publishes zero velocity to stop first, then exits with <code>os._exit(0)</code>

Mouse operation: press `r` and drag to box-select the target color region; on release, the HSV range inside the box is computed automatically (with the `h/s/v_expand` expansion margin) and persisted.

On-screen overlay: state/frame rate/target coordinates, PTZ `s1/s2` angles, body PD parameters, target radius/target distance, and the lost-frame count.

Safety and Edge Cases:

- With no HSV threshold, wait for the user to press `r` and box-pick the color

- Consecutive lost frames < 5 keep the target state unchanged; ≥ 5 frames mark it lost and freeze the body
- Switching to idle resets the PD and publishes a zero `/cmd_vel`
- `s1` is zeroed within `angular_tolerance_deg` (3°), avoiding continuous fine-tuning caused by tiny errors
- Pan-tilt angle limiting: horizontal $[-90^\circ, 90^\circ]$, tilt $[-90^\circ, 20^\circ]$
- Body velocity limiting: angular velocity ± 0.15 rad/s, linear velocity ± 0.12 m/s
- On exit, stop first, then `os._exit(0)`

5. How to Stop

Press `Ctrl+C` to stop Terminal 2 $\rightarrow$ Terminal 1 in order.

4. Experiment Observations

- **idle state (after startup):** The frame streams in real time, the pan-tilt does not move, the body is stationary (publishes a zero `/cmd_vel`), waiting for the user to press Space or `r` to pick the color
- **After color box-picking + tracking:** The pan-tilt locks onto the target center with PD, the body turns to home automatically with the `s1` angle, and simultaneously moves forward/back according to the target size to maintain distance
- **Target centered (`s1` within tolerance):** The body does not turn, only the pan-tilt fine-tunes, and the body linear velocity is controlled by distance following
- **Target deviates at a large angle:** The body takes on the `body_share` (0.4) ratio of the turning task, the pan-tilt takes the rest, and while the body turns the pan-tilt compensates in the opposite direction (`body_ff_gain`)
- **Target distance appropriate:** the radius is within `[target_radius  $\pm$  tolerance]` and the linear velocity is zeroed
- **Target too far (small radius):** the car moves forward until the target radius recovers to the target value
- **Target too close (large radius):** the car moves back to maintain a safe distance
- **Target briefly lost (< 5 frames):** the lost-frame counter increments but does not reach the threshold; the previous valid target state is kept
- **Target lost by timeout (≥ 5 frames):** marked lost, all PDs reset, and the body freezes immediately
- **Press Space to pause:** switches to idle, the body stops immediately, and following can be resumed by pressing Space again

5. Program Analysis

Source code paths:

- Launch:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/color_follow_ptz.launch.py`
- Node:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/color_follow_ptz_node.py`

1. Core Node Analysis

`ColorFollowPtzNode` inherits from `rcipy.node.Node`; `__init__` completes the parameter declaration, subscriber/publisher creation, and the dual-timer initialization:

```
# Source file:
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/color_follow_ptz_node.py

class ColorFollowPtzNode(Node):

    def __init__(self) -> None:
        super().__init__(DEFAULT_NODE_NAME)
        self._tgt_lock = threading.Lock()
        self._cv_bridge = CvBridge()
        self._latest_frame: Optional[np.ndarray] = None

        self._state = STATE_IDLE
        self._shutdown = False

        # Target state
        self._has_target = False
        self._lost_frames = 0; self._lost_frame_threshold = 5
        self._target_x: Optional[float] = None
        self._target_y: Optional[float] = None
        self._target_r: Optional[float] = None

        # PTZ
        self._h_deg = 0.0; self._p_deg = 0.0
        self._prev_err_x = 0.0; self._prev_err_y = 0.0
        self._prev_ctrl_t = 0.0; self._last_sx = 0.0; self._last_sy = 0.0

        # HSV + ROI
        self._hsv = ((0, 85, 126), (9, 253, 253))
        self._mouse_pressed = False; self._mouse_start = (0, 0); self._mouse_end = (0, 0)
        self._roi_ready = False

        # Body PD state
        self._prev_s1 = 0.0; self._prev_body_t = 0.0; self._body_share = 0.5
        self._prev_s1_eff = 0.0
        self._last_body_az = 0.0; self._body_ff_gain = 1.3

        # Distance-follow state
        self._target_radius = 80.0
        self._prev_r_err = 0.0; self._prev_linear_t = 0.0

        self._fps = 0.0; self._fps_n = 0; self._fps_t = time.perf_counter()
        self._win_ok = False; self._last_print_t = 0.0

        self._declare_params()
        self._load_params()
        self._h_deg = self._init_h_deg; self._p_deg = self._init_p_deg

        # Load the HSV from the file
        saved = read_hsv_file(self._hsv_file)
        if saved:
```

```

        self._hsv = saved
        self._apply_hsv_to_params(saved)
        self.get_logger().info(f"Loaded HSV from file: {self._hsv_file}")

    video_qos = QoSProfile(
        history=QoSHistoryPolicy.KEEP_LAST,
        depth=1,
        reliability=QoSReliabilityPolicy.BEST_EFFORT,
        durability=QSDurabilityPolicy.VOLATILE
    )

    self._sub = self.create_subscription(
        Image, self._img_topic, self._on_image, video_qos,
    )
    self._ptz_pub = self.create_publisher(Vector3, self._ptz_topic, 10)
    self._cmd_pub = self.create_publisher(Twist, self._cmd_topic, 10)

    # Timers
    self._ptz_timer = self.create_timer(
        1.0 / max(self._ptz_rate, MINIMUM_TIMER_RATE_HZ), self._ptz_tick,
    )
    self._body_timer = None; self._rebuild_body_timer(False)
    self._key_timer = None
    if self._show_win:
        self._init_win()
        self._key_timer = self.create_timer(0.05, self._poll_keys)

    self._init_ptz_timer = None; self._init_ptz_remain = 0
    if self._pub_init: self._start_init_ptz()

    self._param_cb =
self.add_on_set_parameters_callback(self._on_param_update)
        self.get_logger().info(f"Color follow PTZ+body started. r=ROI,
SPACE=toggle, i=PTZonly, q=quit. "
                                f"ptz={self._ptz_rate:.0f}Hz body=
{self._body_rate:.0f}Hz")

```

HSV target detection method `_detect`: converts the BGR frame to HSV, binarizes it with `cv.inRange`, removes noise with morphological open/close operations, extracts the largest contour and computes the minimum enclosing circle; a valid target updates the coordinates and radius and resets the lost-frame counter.

PTZ control method `_ptz_tick`: normalizes the pixel error, plugs it into the PD formula to compute the step, adds the angular-rate feedforward compensation for the body rotation, and publishes `/cmd_ptz_angle` after two-step EMA smoothing (step smoothing $\alpha=0.35$ + angle smoothing $\alpha=0.85$).

Body control method `_body_tick`: reads the pan-tilt horizontal angle `s1`, multiplies it by `body_share` for the effective error, PD computes the angular velocity; simultaneously PD computes the linear velocity from the deviation between the target enclosing-circle radius and `target_radius`, and publishes the merged `/cmd_vel`.

2. Key Parameters

Parameter definition file:

```
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/color_follow_ptz.launch.py
```

Parameter	Type	Default Value	Description
pd_kp / pd_kd	float	0.7 / 0.25	Pan-tilt PD proportional/derivative gains
ptz_control_rate	float	30.0 Hz	Pan-tilt control rate
body_control_rate	float	10.0 Hz	Body control rate
deadband_x / deadband_y	float	30.0 / 30.0 px	Pan-tilt horizontal/vertical deadband
max_ptz_step	float	1.2 °	Maximum single pan-tilt angle step
ptz_step_smooth_alpha	float	0.35	Step EMA smoothing coefficient
ptz_smooth_alpha	float	0.85	Pan-tilt angle EMA smoothing coefficient
angular_kp / angular_kd	float	0.05 / 0.05	Body angle PD gains
body_share	float	0.4	Ratio of the homing error taken on by the body [0, 1]
body_ff_gain	float	1.3	Angular-rate feedforward gain [0, 2]
angular_tolerance_deg	float	3.0 °	Body angle deadband
max_angular_z	float	0.15 rad/s	Maximum body turning angular velocity
reverse_body_direction	bool	true	Reverse the body turning direction
enable_distance_follow	bool	true	Distance-follow switch
target_radius	float	100.0 px	Distance-follow target radius
distance_kp / distance_kd	float	0.008 / 0.003	Distance-follow PD gains
max_linear_x	float	0.12 m/s	Maximum body linear velocity
distance_tolerance_r	float	12.0 px	Distance-follow deadband
lost_frame_threshold	int	5	Lost-frame tolerance threshold (≥1)

6. Frequently Asked Questions

Problem	Cause	Solution
The target flies out when the body turns too fast	<code>max_angular_z</code> too large or <code>body_share</code> too large	Reduce <code>max_angular_z</code> (e.g., 0.08) and reduce <code>body_share</code> (e.g., 0.2)
The pan-tilt does not compensate for the body rotation	<code>body_ff_gain</code> too small	Increase <code>body_ff_gain</code> (e.g., 1.5)
The distance oscillates far and near	<code>distance_kp</code> too large causes overshoot	Reduce <code>distance_kp</code> (e.g., 0.004)
Target not detected	The HSV threshold is inaccurate	Press <code>r</code> to re-box-pick the target color under the current lighting
The target does not move but the body jitters	<code>angular_tolerance_deg</code> too small	Increase the angle deadband (e.g., 5°)